

05_ 堆的定义数组表示与操作

05 堆的定义、数组表示与操作

题目原文

了解堆的定义、如何用数组来表示堆和实现堆上的操作。

考查类型

概念题和算法设计题。

堆的定义

堆是一棵满足堆序性质的完全二叉树。

最大堆：

每个结点的值 $\geq$ 它的子结点的值

最小堆：

每个结点的值 $\leq$ 它的子结点的值

完全二叉树保证堆可以紧凑地存入数组，不需要显式保存指针。

数组表示

若数组从下标 1 开始：

$$parent(i) = \lfloor i/2 \rfloor$$

$$left(i) = 2i$$

$$right(i) = 2i + 1$$

若数组从下标 0 开始：

$$parent(i) = \lfloor (i - 1)/2 \rfloor$$

$$left(i) = 2i + 1$$

$$right(i) = 2i + 2$$

考试中要先说明使用哪一种下标体系，再写公式。

主要操作

向上调整

插入新元素时，把元素放在数组末尾。若它破坏堆序性质，就不断与父结点交换，直到堆序恢复。

时间复杂度：

$$O(\log n)$$

向下调整

删除堆顶时，用最后一个元素补到堆顶。若它破坏堆序性质，就不断与更合适的子结点交换，直到堆序恢复。

最大堆中应与较大的子结点交换。最小堆中应与较小的子结点交换。

时间复杂度：

$$O(\log n)$$

建堆

自底向上对所有非叶子结点做向下调整。

最后一个非叶子结点的位置为：

$$\lfloor n/2 \rfloor$$

这里采用 1 开始的数组下标。

总时间复杂度：

$$O(n)$$

典型伪代码

最大堆的向下调整：

MaxHeapify(A, i, heapSize):

$$l = left(i)$$

$$r = right(i)$$

$$largest = i$$

```

if  $l \leq \text{heapSize}$  and  $A[l] > A[\text{largest}]$ :
     $\text{largest} = l$ 

if  $r \leq \text{heapSize}$  and  $A[r] > A[\text{largest}]$ :
     $\text{largest} = r$ 

if  $\text{largest} \neq i$ :
    swap  $A[i], A[\text{largest}]$ 
    MaxHeapify( $A, \text{largest}, \text{heapSize}$ )

```

常见应用

优先队列可以用堆实现。

堆排序可以用最大堆实现：先建最大堆，再反复取出堆顶最大元素。

例题

例题 1

题目：判断数组是否表示一个最大堆。数组从下标 1 开始：

$A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

解答：

逐个检查非叶子结点：

$$16 \geq 14, 10$$

$$14 \geq 8, 7$$

$$10 \geq 9, 3$$

$$8 \geq 2, 4$$

$$7 \geq 1$$

每个父结点都不小于子结点，所以该数组表示一个最大堆。

例题 2

题目：向上面的最大堆插入 15，写出调整后的数组。

解答：

先把 15 放到数组末尾：

$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1, 15]$

15 的父结点是 7，交换：

[16, 14, 10, 8, 15, 9, 3, 2, 4, 1, 7]

15 的新父结点是 14，继续交换：

[16, 15, 10, 8, 14, 9, 3, 2, 4, 1, 7]

此时 $15 \leq 16$ ，调整结束。

例题 3

题目：从原最大堆删除堆顶 16，写出调整后的数组。

解答：

用末尾元素 1 补到堆顶，再向下调整：

[1, 14, 10, 8, 7, 9, 3, 2, 4]

[14, 1, 10, 8, 7, 9, 3, 2, 4]

[14, 8, 10, 1, 7, 9, 3, 2, 4]

[14, 8, 10, 4, 7, 9, 3, 2, 1]

最终最大堆为：

[14, 8, 10, 4, 7, 9, 3, 2, 1]

常见误区

堆不是二叉搜索树。堆只保证父子之间的大小关系，不保证左子树所有元素都小于右子树所有元素。

建堆不是 $O(n \log n)$ 的紧分析结果。逐个插入建堆是 $O(n \log n)$ ，自底向上建堆是 $O(n)$ 。